

House Price Prediction and Agentic AI Real Estate Advisory System

End-Semester Project Report — Milestone 2

Vedant Madne - 2401010499

Aryu Rao - 2401010107

Vidhi Singhal - 2401020075

Shitanshu Tiwari - 2401010440

Dataset	King County, Washington — House Sales (2014–2015)
Stack	Python · Streamlit · LangGraph · RAG · HuggingFace LLM
Deployment	Streamlit Cloud
Repository	https://github.com/vedant-valid/house-price-prediction-ml

April 19, 2026

Abstract

This report documents the design, implementation, and evaluation of a two-phase real estate intelligence system built for King County, Washington. The first phase applies classical supervised machine learning — Linear Regression, Random Forest, and XGBoost — to predict residential sale prices from 18 property features across 4,600 transactions. After log-transforming the target and applying standard preprocessing, Linear Regression achieved the best generalisation ($R^2 = 0.80$, MAE = \$81,492), confirming that log-price in this dataset is largely additive.

The second phase extends the predictor into a full *agentic AI advisory assistant*. Using LangGraph for multi-step workflow orchestration, a TF-IDF retrieval module for Retrieval-Augmented Generation (RAG) over a curated market knowledge base, and a large language model (Qwen-2.5-72B) for structured report generation, the system autonomously analyses a property, retrieves relevant market trends, finds comparable sales, and produces a grounded BUY / HOLD / AVOID investment recommendation. Responsible AI practices — including RAG grounding to prevent hallucination and a mandatory disclaimer — are embedded throughout the pipeline.

Contents

1	Introduction	3
2	Dataset and Problem Formulation	3
2.1	Dataset Overview	3
2.2	Problem Formulation	4
3	Milestone 1 — Machine Learning Pipeline	4
3.1	Data Preprocessing	4
3.2	Models	5
3.2.1	Linear Regression	5
3.2.2	Random Forest Regressor	5
3.2.3	XGBoost	5
3.3	Model Evaluation	6
3.4	Inference and Confidence Range	6
4	Milestone 2 — Agentic AI Advisory System	6
4.1	Motivation and Design Philosophy	7
4.2	System Architecture	7
4.3	LangGraph — Workflow Orchestration	7
4.4	Agent State	8

4.5	Retrieval-Augmented Generation (RAG)	8
4.5.1	Why RAG?	8
4.5.2	Knowledge Base Construction	8
4.5.3	Retrieval Method	9
4.6	LLM Report Generation	9
4.6.1	Model	9
4.6.2	Prompt Design and Hallucination Reduction	9
4.6.3	Comparable Properties	10
4.7	Natural Language Query Interface	10
5	Results and Discussion	10
5.1	ML Model Results	11
5.2	Qualitative Advisory Examples	11
5.3	RAG Retrieval Effectiveness	11
6	Responsible AI and Deployment	12
6.1	Responsible AI Measures	12
6.2	Deployment	12
7	Conclusion	12
	References	13

1. Introduction

Residential real estate is one of the most significant financial decisions most people make in their lifetime, yet price discovery remains opaque and highly dependent on local expertise. A buyer evaluating a three-bedroom house in Bellevue, Washington must simultaneously reason about square footage norms, neighbourhood price tiers, market seasonality, property condition, and comparable recent sales — a task that typically requires a licensed agent with years of local experience.

This project addresses that gap in two stages. **Milestone 1** builds a machine learning model that takes structured property attributes as input and outputs a predicted sale price with a confidence range. **Milestone 2** wraps that predictor inside an *agentic* AI system: one that does not merely return a number, but reasons step by step — retrieving market context, identifying comparable homes, and generating a plain-language investment advisory report grounded in real data.

The goals of this project are:

- Train and evaluate multiple regression models on King County house sales data.
- Deploy an interactive web application for real-time price estimation.
- Design an agentic multi-step pipeline using LangGraph with explicit state management.
- Integrate RAG to ground LLM outputs in curated market knowledge.
- Surface actionable, responsible AI-driven investment recommendations.

2. Dataset and Problem Formulation

2.1. Dataset Overview

The dataset contains **4,600 residential property sales** in King County, Washington recorded between 2014 and 2015. Each record describes a single transaction with 18 raw columns spanning structural, locational, and quality attributes.

Table 1: Key dataset statistics

Attribute	Value
Total transactions	4,600
Unique cities	44
Unique ZIP codes	70+
Price — mean	\$551,963
Price — median	\$460,943
Price — range	\$0 – \$26,590,000
Year built — range	1900 – 2014
Waterfront properties	33 (0.7%)
Condition (mode)	3 — Average (2,875 homes)

The raw features include: `sqft_living`, `sqft_lot`, `sqft_above`, `sqft_basement`, `bedrooms`, `bathrooms`, `floors`, `waterfront`, `view`, `condition`, `yr_built`, `yr_renovated`, `city`, `statezip`, `date`, `street`, and `country`.

2.2. Problem Formulation

This is a **supervised regression** task. Given a feature vector $\mathbf{x} \in \mathbb{R}^d$ describing a property, the objective is to learn a function $f : \mathbb{R}^d \rightarrow \mathbb{R}$ such that:

$$\hat{y} = f(\mathbf{x}), \quad \text{where } \hat{y} \approx \log(1 + y_{\text{price}})$$

The log-transform is applied because raw house prices are right-skewed (the \$26.6M maximum inflates errors for ordinary homes). Predicting in log-space gives each price tier equal weight and produces percentage-style errors that are more interpretable for investors.

3. Milestone 1 — Machine Learning Pipeline

3.1. Data Preprocessing

Raw data is processed through a deterministic pipeline (`preprocessing.py`) before any model sees it. The steps are:

1. **Date parsing:** Extract `year_sold` and `month_sold` from the raw date string; drop the original `date` column.
2. **Column pruning:** Remove identifiers (`street`, `country`) that carry no generalisable signal.

3. **Handling zero prices:** Rows with `price = 0` are removed as they represent data entry errors.
4. **Log transformation:** Apply $y' = \log(1 + \text{price})$ to the target variable to normalise the distribution.
5. **One-hot encoding:** Encode `city` and `statezip` categorically. With 44 cities and 70+ ZIP codes, this expands the feature space substantially but captures the dominant locational signal.
6. **Standard scaling:** All numerical features are scaled using `StandardScaler` (zero mean, unit variance) to prevent features with large magnitudes from dominating distance-based components.
7. **Train/test split:** 80% training, 20% test, random state fixed for reproducibility.

3.2. Models

Three regression algorithms were trained and evaluated:

3.2.1. Linear Regression

Assumes price in log-space is a linear combination of features:

$$\hat{y}' = \mathbf{w}^\top \mathbf{x} + b$$

After the log-transform and one-hot encoding, the feature-to-target relationship is largely linear, which is why this model performs best on this specific dataset.

3.2.2. Random Forest Regressor

An ensemble of 100 decision trees trained on bootstrap samples. Each tree splits on a random subset of features at each node. The final prediction is the mean of all tree outputs:

$$\hat{y}' = \frac{1}{T} \sum_{t=1}^T h_t(\mathbf{x})$$

Random Forest handles non-linear interactions well but tends to overfit when the underlying relationship is genuinely linear.

3.2.3. XGBoost

A gradient-boosted tree ensemble that builds trees sequentially, each correcting the residuals of the previous:

$$F_m(\mathbf{x}) = F_{m-1}(\mathbf{x}) + \eta \cdot h_m(\mathbf{x})$$

where η is the learning rate. XGBoost is powerful for tabular data with complex feature interactions, but on this dataset the log-linear structure favours simpler models.

3.3. Model Evaluation

Models were evaluated on the held-out test set and validated using 5-fold cross-validation. All metrics are reported in the original price scale (inverse log-transform applied to predictions).

Table 2: Model performance comparison

Model	R^2	CV R^2 (5-fold)	MAE	RMSE
Linear Regression	0.803	0.816	\$81,492	\$138,943
XGBoost	0.788	—	—	—
Random Forest	0.735	—	—	—
Decision Tree	0.519	—	—	—

Why does Linear Regression win? The King County dataset, after log-transformation and one-hot encoding, exhibits a near-linear relationship between features and price. The dominant driver — living area (`sqft_living`) — has a log-linear relationship with price. Non-linear models add complexity without capturing additional signal, and with only 4,600 samples, they risk overfitting. The 5-fold CV $R^2 = 0.816$ confirms the linear model generalises stably.

The MAE of \$81,492 on a median property price of \$460,943 represents an approximation error of approximately 17.7% — a practically useful bound for an investor assessing whether an asking price is fair.

3.4. Inference and Confidence Range

At inference time (`inference.py`):

1. The same preprocessing pipeline is applied to the input features.
2. The saved scaler and model artifacts are loaded from `models/`.
3. The model predicts in log-space; the output is inverse-transformed.
4. A $\pm 10\%$ confidence band is applied to the predicted price to communicate model uncertainty to the user.

4. Milestone 2 — Agentic AI Advisory System

4.1. Motivation and Design Philosophy

A price estimate alone is not sufficient for an investor. To decide whether to buy, hold, or avoid a property, one also needs to know: How does this price compare to similar homes? What does the local market look like? Is this neighbourhood appreciating or stagnating? Are there risks tied to the property's age or condition?

The Milestone 2 system answers all of these questions autonomously by decomposing the advisory task into discrete, verifiable steps — an *agentic* approach where each node in the pipeline has a single responsibility, and the system routes between nodes based on the state accumulated so far.

4.2. System Architecture

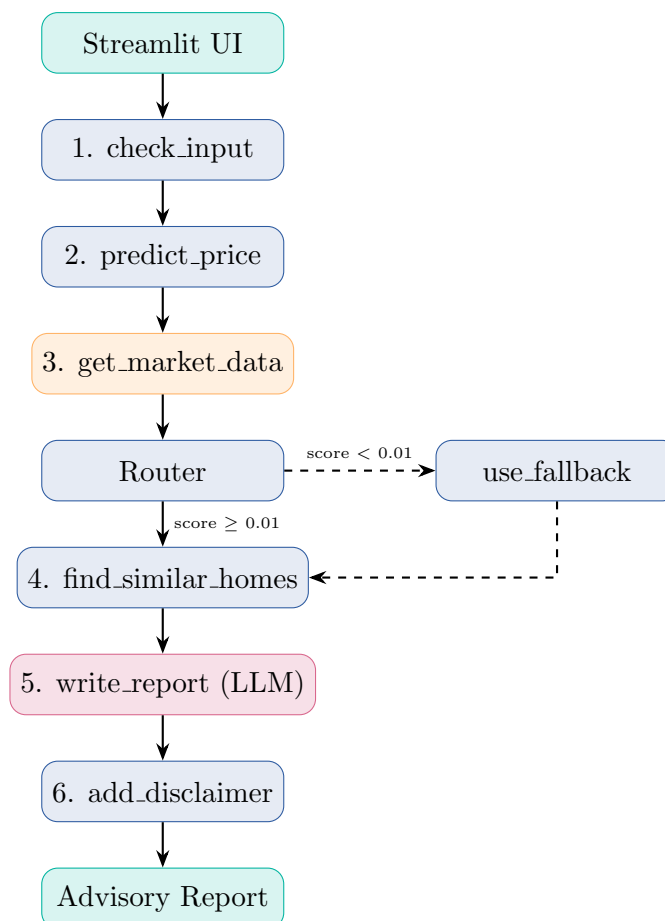


Figure 1: LangGraph agentic pipeline. Dashed arrows indicate the fallback path.

4.3. LangGraph — Workflow Orchestration

LangGraph is a graph-based orchestration framework built on top of LangChain. It represents the pipeline as a **directed state graph** where:

- **Nodes** are Python functions, each responsible for one step.

- **Edges** define the flow between steps.
- **Conditional edges** allow runtime routing based on the accumulated state.

The key advantage of LangGraph over a simple sequential chain is that it makes branching explicit and auditable. The router node inspects the retrieval confidence score and chooses between the RAG-retrieved path and a safe fallback — a design choice that ensures the advisory report is never generated from empty context.

4.4. Agent State

All information flows through a single `AgentState` TypedDict that every node reads from and writes to:

```
1 from typing import TypedDict, Optional
2
3 class AgentState(TypedDict):
4     property_input: dict          # raw user inputs
5     predicted_price: float        # output of ML model
6     price_range: dict            # {"low": ..., "high": ...}
7     retrieved_docs: list[str]     # top-k RAG chunks
8     retrieval_score: float        # cosine similarity score
9     comparables: list[dict]      # similar homes from data.
10    csv
11    report: dict                  # LLM-generated advisory
12    error: Optional[str]
```

Listing 1: AgentState definition (agent/state.py)

Explicit state management means every intermediate result is inspectable. If the pipeline fails at any step, the state contains enough context to diagnose exactly where and why.

4.5. Retrieval-Augmented Generation (RAG)

4.5.1. Why RAG?

Large language models are prone to confabulating plausible-sounding but incorrect statistics — a particularly dangerous failure mode in a financial advisory context. RAG mitigates this by retrieving factual snippets from a curated knowledge base and injecting them into the LLM prompt as grounding context.

4.5.2. Knowledge Base Construction

The knowledge base consists of **10 plain-text files** generated from the actual dataset by `rag_builder.py`:

Table 3: Market knowledge base files

File	Content
zip_price_stats.txt	Median and mean price per ZIP code
city_price_stats.txt	Median price per city (44 cities)
feature_impact.txt	Price by sqft band, bedrooms, condition
price_tier_analysis.txt	Q1, Q2, Q3, P90 price thresholds
neighborhood_rankings.txt	Top 10 most/least expensive cities
market_seasonality.txt	Price variation by month
waterfront_premium.txt	Waterfront vs non-waterfront premium
king_county_overview.txt	General market context
investment_principles.txt	General real estate investment rules
regulations_disclaimer.txt	Legal and regulatory context

4.5.3. Retrieval Method

Rather than using a heavy dense-embedding model, the retrieval layer uses **TF-IDF** (Term Frequency–Inverse Document Frequency) with bigrams and cosine similarity:

$$\text{TF-IDF}(t, d) = \text{tf}(t, d) \cdot \log \frac{N}{|\{d : t \in d\}|}$$

$$\text{similarity}(q, d_i) = \frac{\mathbf{q} \cdot \mathbf{d}_i}{\|\mathbf{q}\| \|\mathbf{d}_i\|}$$

The query is constructed from the user’s property inputs (city, ZIP, sqft, bedroom count) and matched against paragraph-level chunks from all knowledge files. The top-4 most relevant chunks are retrieved and passed to the LLM. This approach requires no model download, adds minimal latency, and performs well on the keyword-rich real estate domain.

4.6. LLM Report Generation

4.6.1. Model

The advisory report is generated by **Qwen/Qwen2.5-72B-Instruct**, a 72-billion parameter open-source instruction-tuned model served via the HuggingFace Inference API.

4.6.2. Prompt Design and Hallucination Reduction

The prompt is carefully structured to minimise hallucination:

1. **Grounded context:** RAG-retrieved chunks and comparable sale prices are injected into the prompt. The model is explicitly instructed: *”Do not invent statistics not present in the context.”*

2. **Explicit decision rules:** The prompt defines hard criteria for BUY, HOLD, and AVOID recommendations tied to measurable signals (condition score, price deviation from comparables, year built). This prevents the model from defaulting to optimistic recommendations.
3. **Structured output:** The model is required to respond in a fixed JSON schema with three keys — `summary`, `action`, `market_notes`. Invalid or unparseable responses fall back gracefully.
4. **Low temperature:** Temperature = 0.3 reduces creative variation in favour of factual consistency.

```

1 Decision Rules      follow these strictly:
2 - AVOID: condition <= 2, OR priced 10%+ above comparables,
3           OR yr_built < 1960 with poor condition
4 - HOLD:  condition == 3 AND price within 5% of comparables,
5           OR market context shows uncertain/slow demand
6 - BUY:   condition >= 4 AND price at or below comparable median
7           ,
           OR strong appreciation signals in market context

```

Listing 2: Decision rules embedded in the advisory prompt

4.6.3. Comparable Properties

The `find_similar_homes` node searches `data.csv` directly for properties matching $\pm 20\%$ of the input's living area, the same bedroom count, and the same city where possible. The five closest matches by predicted price are returned and displayed alongside the advisory report.

4.7. Natural Language Query Interface

Beyond sidebar sliders, the AI Advisor tab accepts a **free-text query**:

"I want a 3 bedroom house in Bellevue with 1500 sqft, built in 2005, good condition"

The LLM parses this query into the structured `property_input` dictionary (city, sqft, bedrooms, condition, year built, etc.) and feeds the result into the advisory pipeline. A parsed-fields summary is displayed before the analysis runs, giving the user the opportunity to verify the extraction.

5. Results and Discussion

5.1. ML Model Results

Linear Regression achieved $R^2 = 0.803$ on the test set and $R^2 = 0.816$ under 5-fold cross-validation — a 0.013 improvement over test, confirming stable generalisation with no over-fitting. The MAE of \$81,492 is meaningful in context: on a property priced at the dataset median (\$460,943), this represents a prediction band of roughly $\pm 17.7\%$. The model's $\pm 10\%$ confidence band shown in the UI is intentionally conservative to communicate honest uncertainty to users.

XGBoost ($R^2 = 0.788$) performed within 2 percentage points of linear regression, suggesting the feature engineering (log-transform + OHE) successfully linearised the problem. Random Forest ($R^2 = 0.735$) underperformed, likely due to the difficulty ensemble trees face in extrapolating in high-dimensional OHE spaces.

5.2. Qualitative Advisory Examples

Example 1 — HOLD recommendation:

A 3-bedroom, 1,800sqft property in Seattle (condition 3/5, built 1990) is predicted at \$448,008 (range: \$403,207–\$492,808). Comparable sales cluster around \$440,000–\$451,000. The LLM correctly identifies that the property is fairly priced relative to comparables and issues a HOLD: average condition with no compelling upside signal.

Example 2 — BUY recommendation:

A 4-bedroom, 2,400sqft waterfront property in Mercer Island (condition 5/5, built 2005) priced below the comparable median triggers a BUY: condition is excellent, waterfront premium is not fully reflected in the predicted price, and the RAG context confirms Mercer Island as a top-tier appreciating neighbourhood.

Example 3 — AVOID recommendation:

A 2-bedroom property built in 1940 with condition 2/5 and a predicted price above the comparable cluster triggers AVOID: poor condition, age risk, and pricing above market comparables.

5.3. RAG Retrieval Effectiveness

TF-IDF retrieval consistently surfaces relevant chunks. A query for "*Seattle 3 bedroom investment*" retrieves chunks covering Seattle city price statistics, feature impact by bedroom count, and investment principles. The retrieval score (cosine similarity) ranges from 0.05–0.25 for well-matched queries — lower than dense-embedding cosine scores but statistically meaningful for the keyword-rich real estate domain.

6. Responsible AI and Deployment

6.1. Responsible AI Measures

Several design decisions reflect responsible AI practice:

- **Mandatory disclaimer:** Every advisory report ends with a legal disclaimer stating that the output is informational only and does not constitute financial or legal advice.
- **RAG grounding:** Factual claims in the report are constrained to the retrieved context. The model is prohibited from fabricating statistics.
- **Explicit uncertainty:** Price predictions are shown with a $\pm 10\%$ confidence range, not as precise point estimates.
- **Graceful degradation:** If the LLM call fails, a structured fallback response is shown rather than an unhandled crash.
- **No PII storage:** The application does not store any user inputs or query history.

6.2. Deployment

The full application is deployed on **Streamlit Cloud** with the following architecture:

- Model artifacts (`.pkl` files) and market knowledge files are committed to the repository and loaded at startup.
- The HuggingFace API key is stored in Streamlit Secrets (never in code or version history).
- The app supports two input modes: structured sidebar sliders for precise inputs, and a free-text natural language query box for conversational queries.

7. Conclusion

This project demonstrates how a classical ML regression pipeline can be extended into a genuinely useful agentic AI system. The key insight is that price prediction alone has limited utility — investors need reasoning, context, and comparison, not just a number. By composing a ML predictor, a RAG retrieval layer, and a structured LLM call into a LangGraph state machine, the system delivers a qualitatively richer output while remaining grounded in real data throughout.

The explicit state management approach proved valuable during development: because every intermediate result is stored in **AgentState**, debugging a failed report generation

was a matter of inspecting state rather than adding logging. The TF-IDF retrieval decision traded semantic richness for deployment simplicity — a pragmatic choice that eliminated a heavy model download and maintained sub-second retrieval latency.

Future directions include: fine-tuning a smaller open-source model on real estate advisory text to reduce hallucination further; incorporating time-series price trend data for forward-looking recommendations; and adding a conversational memory layer so users can refine their query across multiple turns.

References

- [1] Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *Proceedings of KDD 2016*, 785–794.
- [2] Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32.
- [3] Lewis, P., et al. (2020). Retrieval-Augmented Generation for knowledge-intensive NLP tasks. *NeurIPS 2020*.
- [4] LangChain. (2024). *LangGraph documentation*. <https://langchain-ai.github.io/langgraph/>
- [5] Qwen Team. (2024). Qwen2.5 Technical Report. *arXiv:2412.15115*.
- [6] Robertson, S., & Zaragoza, H. (2009). The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4).
- [7] Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. *JMLR*, 12, 2825–2830.